

High-Frequency Direction-of-Arrival Analysis Using Calibration and Kalman Filtering

Dror Matalon and Danel Vardimon

August 29, 2025

Contents

1	Problem discussion	4
2	Signal Model	5
2.1	Introduction	5
2.1.1	Mathematics background-Hermetic Matrix	5
2.2	Phased array-Antennas Properties and Patterns	5
2.2.1	Near Field vs. Far Field	5
2.2.2	Gain of Antennas	6
2.2.3	Array factor	7
2.2.4	Array gain	7
2.2.5	Beamwidth	7
2.2.6	steering vector	8
2.2.7	Isotropic antenna	8
2.2.8	Null Steering	8
2.3	Our Signal Model based on phased array	8
3	DOA	11
3.1	Introduction	11
3.2	DOA estimations Methods	11
3.2.1	Delay-and-Sum Method	11
3.2.2	MVDR Method	12
3.2.3	MUSIC-Multiple Signal Classifier	12
3.2.4	Complexity	12
3.2.5	MDL	13
3.3	Chapter Summary	13
4	Kalman Filter	15
4.1	Introduction	15
4.2	Before diving into Kalman filters	15
4.2.1	statistics	15
4.2.2	Alpha Filter	16
4.2.3	Alpha-Beta Filter	16
4.2.4	Noise Types Definition	17
4.3	Kalman Filter in 1D	17
4.3.1	Background on the Kalman Filter	17
4.3.2	Kalman Filter for a Constant Variable	18
4.3.3	Five Kalman Filter Equations	19

4.3.4	Kalman filter for changing variable	19
4.3.5	Process Noise	20
4.3.6	Lag error	20
4.3.7	Confidence interval	21
4.4	Kalman filter in N-D	21
4.4.1	Formalization of High-Dimensional Kalman Filter	21
4.4.2	State prediction Equation	22
4.4.3	Covariance prediction Equation	23
4.4.4	State Update Equation	23
4.4.5	Covariance Update Equation	24
4.5	What is a non linear system	25
4.6	EKF – Extended Kalman Filter	26
4.6.1	Linear Approximation	26
4.6.2	Observation Matrix	26
4.6.3	Transformation Matrix	27
4.6.4	LKF for Oscillator	28
4.6.5	EKF for Oscillator	28
4.7	Chapter Summary	29
5	Phased and Gain distortion	34
5.1	introduction	34
5.2	Coupling	34
5.2.1	IQ imbalance	34
5.2.2	Couplings model	34
5.3	Minimizing the EVM (Error Vector Magnitude)	35
5.3.1	Reducing the GP mismatch	35
5.3.2	Chapter Summary	36
6	Our System Model	39
6.0.1	System model	39
7	Simulations	41
7.1	Real recordings analysis- Gain and Phase simulation	41
7.2	Swinging pole simulation	42
8	Conclusion	44
8.1	Our insights	44
8.2	For future work	44
	Bibliography	45

List of Figures

2.1	Gain factors comparison	6
2.2	total pattern G	7
2.3	total pattern G polar graph	7
3.1	DOA Methods	13
4.1	Kalman filter stages scheam	18
4.2	Constant velocity kalman filter equations	20
4.3	High Sigma	21
4.4	Low Sigma	21
4.5	High dimentional kalman filter five equations	25
4.6	EKF equations	27
6.1	Final system architecture	39
7.1	Gain mismatch	41
7.2	Phase mismatch	41
7.3	MUSIC Before Calibration	41
7.4	MUSIC After Calibration	41
7.5	Comparison of DOA algorithms	42
7.6	MUSIC performance example	43

Chapter 1

Problem discussion

In recent years, the demand for mobile communications has increased and continues to grow exponentially. To meet this demand, a wider portion of the electromagnetic spectrum is being utilized, and the deployment of smaller cells is rapidly expanding. These small cells offer high throughput and improved coverage in urban centers. However, as the number of deployed small cells increases, the need for efficient backhaul communication between them and the core network becomes a critical challenge [5].

Traditional wired backhaul solutions are impractical in such dense areas due to prohibitive installation and maintenance costs. An effective alternative is to use wireless backhaul at very high frequencies — such as the unlicensed 60 GHz and lightly licensed 70–80 GHz ranges.

Operating at these high frequencies for backhaul allows the use of phased arrays with a larger number of antenna elements, enabling highly focused beams. While such beams provide significant advantages — including higher SNR, increased link capacity, and potentially lower power consumption — they also introduce a new challenge. Even small physical movements of the communication poles or mounting structures carrying these arrays can lead to beam misalignment, resulting in severe link degradation or complete outage.

Approximately, for a uniform linear array (ULA) with M antennas, the angular width of the beam is given by:

$$\Delta\theta = 2 \sin^{-1} \left(\frac{0.891}{M} \right)$$

Using this formula, a 3 dB loss is caused by a misalignment of $0.3578 \Delta\theta$. For example, using $M = 20$ antennas, the misalignment causing 3 dB loss is of 1.827 degrees and for $M = 256$ its around 0.142.

Therefore, precise and reliable antenna alignment becomes essential for maintaining link quality in high-frequency backhaul systems, especially in dense small-cell networks. In this work we examine the usage of Kalman filter in order to handle the misalignment problem.

Chapter 2

Signal Model

2.1 Introduction

In this chapter, we will understand how a phased array works and build a signal model based on it.

2.1.1 Mathematics background-Hermetic Matrix

A Hermitian matrix is a complex square matrix that is equal to its own conjugate transpose. That is:

$$A = A^H$$

Where: A^H means transpose + complex conjugate. All of the eigenvalues of a Hermitian matrix are real — even though the matrix can contain complex numbers. The diagonal entries are always real. an example from signal processing Hermitian matrices often show up as correlation or covariance matrices:

$$R = E[XX^H]$$

2.2 Phased array-Antennas Properties and Patterns

A phased array is a group of antennas arranged in a specific pattern, where each antenna's signal is phase-shifted using a time delay. By controlling these phase shifts, the array can steer the beam of energy in a desired direction without physically rotating the antenna. The following analysis closely follows the approach described in [2]

2.2.1 Near Field vs. Far Field

when we have an emitting source that match a near field condition, the angel varies for each antenna. This makes the detection of direction of arrival more complicated. although in many cases we can make an approximation that the wave comes as straight line and thus all the antennas get the same angle. this apply for Far Field and the condition is:

$$\text{Far Field: } R > \frac{2D^2}{\lambda}$$

2.2.2 Gain of Antennas

To determine the direction from which the antenna receives the most energy, we use the concept of Directivity. An antenna directivity is defined as its radiation power divided by an isotropic antenna pattern (sometimes D referred to as D_{max}):

$$D = \frac{P_{(\theta,\phi)}}{P_{iso}(\theta,\phi)}$$

Another definition that is usually used to describe antenna direction is the antenna gain which takes into account the losses of power in the system:

$$G = D\eta_{rad}$$

Where $\eta_{rad} = \frac{P_{rad}}{P_{in}}$

$$D = \frac{P_{max}}{P_{avg}}$$

Now we can amend the Gain of the antenna

$$\text{Gain} = kD, \quad \text{where} \quad k = \frac{P_{rad}}{P_{in}}$$

We will digress here signals in one dimension and it will be easier to investigate our space using degrees.

$$G(\theta) = G_E(\theta) + G_A(\theta), \text{ in dB} \quad (2.1)$$

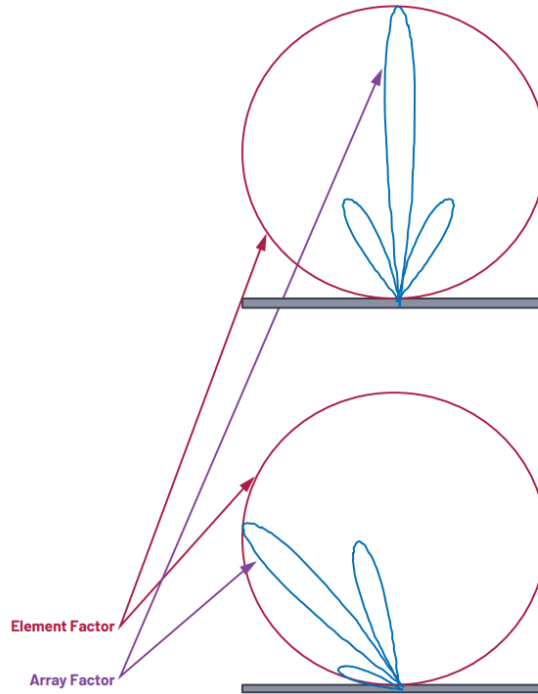


Figure 2.1: Gain factors comparison

2.2.3 Array factor

from Equation 2.1 we will discuss on the $G_A(\theta)$ factor. The array factor comes from our beamforming. In our case we will focus on ULA and the right expression for him is:

$$AF[\theta] = \frac{\sin\left(\frac{N\pi d}{\lambda} [\sin(\theta) - \sin(\theta_0)]\right)}{N \sin\left(\frac{\pi d}{\lambda} [\sin(\theta) - \sin(\theta_0)]\right)}, \theta_0 = \text{beam angle} \quad (2.2)$$

using the connection:

$$\Delta\Phi = \frac{2\pi d \sin \theta}{\lambda} \quad (2.3)$$

we can rewrite the factor 2.2:

$$AF[\theta, \Phi] = \frac{\sin\left(N \left[\frac{\pi d}{\lambda} \sin(\theta) - \frac{\Delta\Phi}{2}\right]\right)}{N \sin\left(\frac{\pi d}{\lambda} \sin(\theta) - \frac{\Delta\Phi}{2}\right)} \quad (2.4)$$

From a quick investigation, we observe that increasing N results in a higher spatial frequency ω , which leads to more oscillations within the angular interval—meaning, we observe more folding.

Additionally, for a larger beam angle θ_0 , the phase Φ increases as shown in Equation 2.3. This phase increase causes a rightward shift in the array factor due to phase subtraction.

2.2.4 Array gain

from Equation 2.1 we will mention that $G_E(\theta)$ factor is created by the geometry of the antenna. we can get a good look at figure 2.2 on the Array gain of the combination between the Element factor and the array factor.

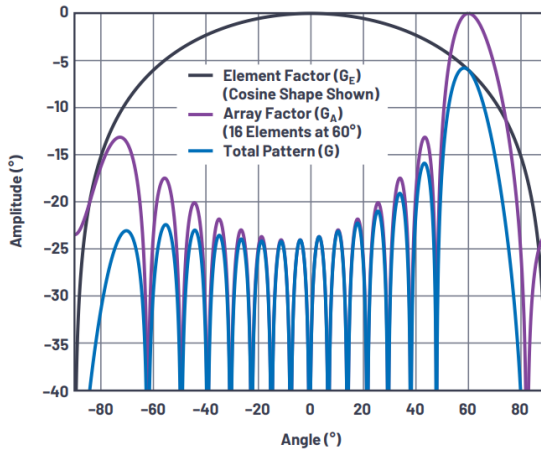


Figure 2.2: total pattern G

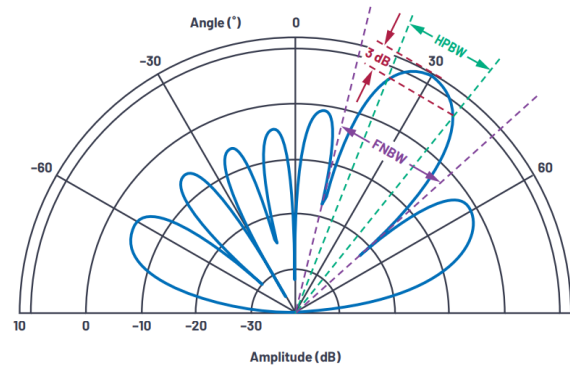


Figure 2.3: total pattern G polar graph

2.2.5 Beamwidth

we can measure the beamwidth by two ways:

- HPBW- The width of the beam at the points of decending 3 dB. we can find the θ by using the Equation 2.4 and make the gain to be $\frac{1}{\sqrt{2}}$

- FNBW-The width of the beam at the folding points. We can calculate like HPBW but the gain would be 0

2.2.6 steering vector

The steering vector is like a set of phase delays or weights that we apply to each antenna signal to make them all "line up" in time as if the signal came straight on. Once aligned, the signals add up coherently, boosting the energy in that direction. In our project we will utilize 4 antennas and a distance of $d = \frac{\lambda}{2}$. the steering Vector is:

$$\mathbf{a}(\theta) = \begin{bmatrix} 1 \\ e^{-j\Delta\phi} \\ e^{-j2\Delta\phi} \\ e^{-j3\Delta\phi} \end{bmatrix} \quad \text{where } \Delta\phi = \frac{2\pi d}{\lambda} \sin(\theta) = \pi \sin(\theta) \quad (2.5)$$

Now we got a steering array for one dimension depends on θ . When we apply these weights and sum the signals, we get constructive interference from that direction and cancel out signals from other directions.

2.2.7 Isotropic antenna

In order to quantify antennas, we usually compare them to an isotropic antenna - which is a hypothetical antenna with equal radiation in every direction. The unit used for such comparison is called dBi and is defined as $dB \left(\frac{P_{\text{rad}}}{P_{\text{isotropic}}} \right)$.

2.2.8 Null Steering

In order to overcome interference, we can use the ability to configure the radiation pattern of the antenna array factor (AF). The concept of choosing a configuration that cancels the radiation from a specific direction is called null steering.

2.3 Our Signal Model based on phased array

After gaining an understanding of the operation of phased arrays, we now present the signal model. To analyze the benefits of the Kalman filter for Direction of Arrival (DOA) estimation, it is necessary to first model the signal received by a Uniform Linear Array (ULA) and examine its properties [3]. Consider a ULA with the following parameters:

- d – distance between adjacent antenna elements.
- λ – wavelength of the received signal.
- θ – angle of arrival (AoA) of the incoming plane wave, measured with respect to the array broadside.

When a narrowband plane wave is transmitted on the array from AoA θ , the propagation path length between adjacent elements differs, resulting in a constant phase shift. This phase difference between two consecutive antennas is given by:

$$\gamma = 2\pi \frac{d}{\lambda} \sin \theta$$

Based on this, we define the *steering vector* for a ULA with M elements as:

$$\mathbf{a}(\theta) = \begin{bmatrix} 1 \\ e^{-j\gamma} \\ e^{-j2\gamma} \\ \vdots \\ e^{-j(M-1)\gamma} \end{bmatrix}$$

Assuming additive white Gaussian noise (AWGN) that is spatially uncorrelated between array elements, the received signal vector can be written as:

$$\mathbf{s} = \sum x_i \mathbf{a}_i(\theta) + \mathbf{n}_i$$

where x is the complex base-band representation of the transmitted signal and $\mathbf{n}$ is the noise vector.

When applying a digital beamformer, the received signal is multiplied by a set of beamforming weight vectors, each designed to enhance signals arriving from a specific direction. If defining a weight matrix:

$$\mathbf{W} = [\mathbf{w}_1 \quad \mathbf{w}_2 \quad \cdots \quad \mathbf{w}_K]$$

where $\mathbf{w}_k$ is the beamforming weight vector for the k -th beam and K is the total number of beams, The beamformer output can then be expressed as:

$$\mathbf{y} = \mathbf{W}^H \mathbf{s}$$

Spatial Correlation Matrix

In array signal processing, an important statistical characterization of the received data is given by the spatial correlation matrix, defined as:

$$\mathbf{R} = E [\mathbf{s}\mathbf{s}^H]$$

Substituting the signal model while assuming orthogonality between steering vectors yields:

$$\mathbf{R} = \sum \sigma_x^2 a(\theta_i) a^H(\theta_i) + \sigma_n^2 \mathbf{I}_M$$

where:

- σ_x^2 is the signal power,
- σ_n^2 is the noise variance,
- $\mathbf{I}_M$ is the $M \times M$ identity matrix.

Since the noise is spatially uncorrelated with the steering vectors, it can be shown that $a(\theta_i)$ are eigenvectors of $\mathbf{R}$ associated with its largest eigenvalues. The eigen-decomposition of $\mathbf{R}$ can be written as:

$$\mathbf{R} = \mathbf{Q}_s \mathbf{\Lambda}_s \mathbf{Q}_s^H + \mathbf{Q}_n \mathbf{\Lambda}_n \mathbf{Q}_n^H$$

where:

- $\mathbf{Q}_s \in C^{M \times P}$ contains the **signal subspace** eigenvectors,
- Λ_s is a $P \times P$ diagonal matrix of the P largest eigenvalues,
- $\mathbf{Q}_n \in C^{M \times (M-P)}$ contains the **noise subspace** eigenvectors,
- Λ_n is a diagonal matrix where all entries are equal to σ_n^2 .

Chapter 3

DOA

3.1 Introduction

In this chapter, we present three algorithms for Direction-of-Arrival (DOA) estimation and discuss the benefits of each. The content of this chapter closely follows the work in [3]. In addition, we introduce a supporting utility algorithm designed to enhance one of the DOA methods.

3.2 DOA estimations Methods

DOA algorithms can be divided into three categories:

- extrema-searching techniques- utilize estimates of its orthogonal complement, often referred to as noise subspace. the three following methods utilize this technique .
- polynomial-rooting techniques- utilize most often estimations like extrema-searching techniques. for example FT-DoA Method utilize this technique.
- matrix-shift techniques- utilize estimates of the signal subspace. for example the ESPRIT Method utilize this technique.

3.2.1 Delay-and-Sum Method

The method calculates the DOA by measuring the signal strength at each possible arrival angle (scanning) and selecting the arrival angles at power peaks. We will multiply the summed signal in a steering vector of our system where the highest weight is the direction of θ_0

$$y(n) = \mathbf{w}^H \mathbf{x}(n) \quad (3.1)$$

Despite being computationally simple, the width and height of the side lobes limit the performance when signals from multiple directions or sources are involved. One way to improve this is to increase the number of sensors, thereby increasing the number of elements in the steering vector $\mathbf{a}(\theta)$, which enhances the delay-and-sum signal

processing. The autocorrelation and Power can be calculated the following way:

$$\hat{\mathbf{R}} = \frac{1}{S} \sum_{n=0}^{S-1} \mathbf{x}_n \mathbf{x}_n^H \quad (3.2)$$

$$P_{\text{DS}}(\theta) = E[|y(n)|^2] = \mathbf{w}^H \mathbf{R} \mathbf{w} = \mathbf{a}^H(\theta) \mathbf{R} \mathbf{a}(\theta) \quad (3.3)$$

where M is the number of antennas, L is the number of sources, S is the number of snapshots (samples), and P is the number of scan steps of $\theta \in [-90^\circ, 90^\circ]$. The highest peaks will give us the direction of the signals.

3.2.2 MVDR Method

It is similar to the delay and sum technique since it evaluates the power of the received signal in all possible directions. We want to minimize the power of the noise and other signals from other directions while it is subject gain of 1 to beamformer in the wanted direction. the solution of the optimization is:

$$P_{\text{MVDR}}(\theta) = \frac{1}{\mathbf{a}^H(\theta) \mathbf{R}^{-1} \mathbf{a}(\theta)} \quad (3.4)$$

The disadvantage of this method is that an inverse matrix computation increase the complexity, however, it provides higher resolution than the delay-and-sum method

3.2.3 MUSIC-Multiple Signal Classifier

Consider the orthogonal connection between the signal and the noise sub-spaces. The method count on it by multiply the steering vector with the eigenvectors of the noise. Due to orthogonal the product should be 0. To avoid errors in estimation $\mathbf{Q}_n$ we will use the following formula:

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta) \mathbf{Q}_n \mathbf{Q}_n^H \mathbf{a}(\theta)} \quad (3.5)$$

We are dividing a constans by an expression that has a limit of 0 and thus we will get a very large power. It is important to mention that it is not the real power and we call it a pseudo "spectrum" provided by MUSIC.

3.2.4 Complexity

the methods Delay-and-Sum and MVDR have pretty much complexity alike, though MUSIC complicity is higher a bit due to inverse matrix calculation complicity. On the other hand the complexity of MUSIC is much higher from both of them but gives a higher resolution.

- Delay-and-Sum- $M^2(S + 2) + M + 4LP$
- MVDR - $M^2(S + 6) + M + 4LP$
- MUSIC $(5/3)M^3 + M^2(S + 1 + L + M) + 4LP$

3.2.5 MDL

As explained in the MUSIC algorithm, it is crucial to determine the number of signals in order to separate the autocorrelation matrix into signal and noise subspaces. To achieve this, we employ the MDL algorithm, following the approach described in [7]. The main idea is to decompose the autocorrelation matrix into eigenvalues and then distinguish between those associated with signals and those associated with noise. This is done by identifying eigenvalues above a certain threshold, which are counted as signal-related, while the remaining ones are considered noise. The performance of the MDL algorithm is sensitive to the signal-to-noise ratio (SNR): a higher SNR provides greater certainty in distinguishing between signal and noise eigenvalues.

3.3 Chapter Summary

The MUSIC algorithm provides the best separation of signal directions, but it requires prior knowledge of the number of signals present, and the MDL criterion used to estimate this number may miss signals depending on the signal-to-noise ratio (SNR). In contrast, the Delay-and-Sum method offers the lowest computational complexity, while the MVDR method performs better in low-SNR conditions. Unlike MUSIC, both Delay-and-Sum and MVDR do not rely on MDL for operation.

A comparison of the three DOA estimation methods discussed in this chapter is presented below, illustrating their performance in the presence of both a desired signal and an interfering signal:

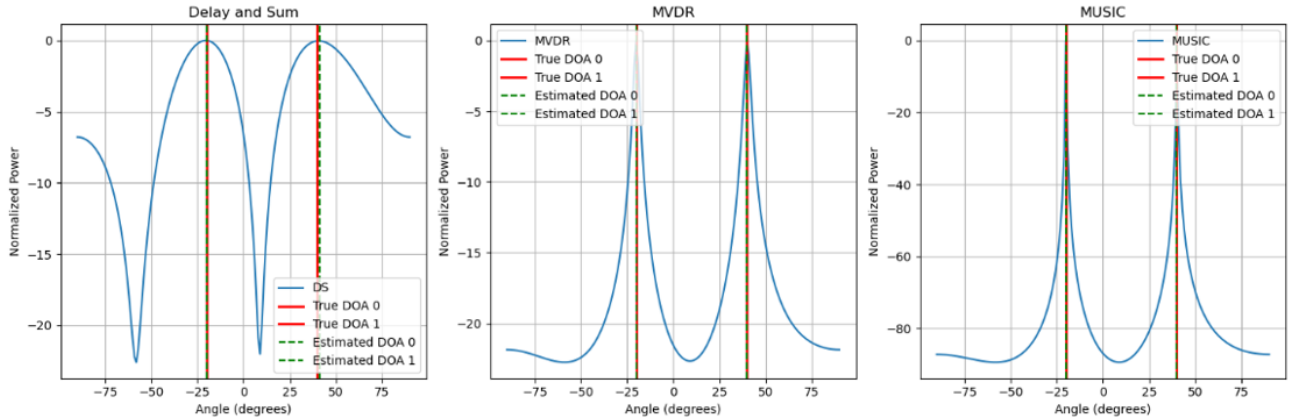


Figure 3.1: DOA Methods

The implementation of the MDL algorithm used in our analysis is provided below:

```

class MdlForSigCnt:
    def __init__(self, ant_num, snapshots_for_r_calc, regulationPower):
        self.mAntNum = ant_num
        self.mSnapForRCalc = snapshots_for_r_calc
        self.mregulationPower=regulationPower

    def GetSamples(self, R):
        eigvals, eigvecs = np.linalg.eigh(R)
        eigvals = np.flip(np.sort(eigvals))
        #flip to decent order
        MdlVal = np.zeros(self.mAntNum - 1)
        for sig_vec_num in range(self.mAntNum - 1):
            noise_eigvals = eigvals[sig_vec_num:]
            gm = np.prod(noise_eigvals) ** (1 / len(noise_eigvals)) #
            #geometric mean
            am = np.mean(noise_eigvals) #
            #arithmetic mean
            if gm <= 0 or am <= 0:
                MdlVal[sig_vec_num] = np.inf
            else:
                regulation = 0.5 * sig_vec_num * (2 * self.mAntNum -
sig_vec_num) * np.log(self.mSnapForRCalc)
                data_term = -self.mSnapForRCalc * (self.mAntNum -
sig_vec_num) * np.log(gm / am)
                MdlVal[sig_vec_num] = data_term +
regulation*self.mregulationPower
            EstSigVecsCnt = np.argmin(MdlVal)

            if EstSigVecsCnt == 0:
                print("0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o")
                print("MDL - estimated signal count is 0")
                print("0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o0o")
                exit()

        return EstSigVecsCnt

```

Chapter 4

Kalman Filter

4.1 Introduction

State estimation is a fundamental problem in control and signal processing, and a variety of filtering algorithms have been developed to address different use cases. This chapter introduces the Kalman filter as a principled recursive estimator for linear Gaussian systems and then generalizes the discussion to the Extended Kalman Filter (EKF) for weakly nonlinear systems. We derive the Kalman equations step by step, explain the underlying system and measurement models, and examine the treatment of process and measurement uncertainty. The presentation in this chapter is based primarily on Becker's book [1].

4.2 Before diving into Kalman filters

4.2.1 statistics

Before diving into Kalman filters, there are several basic definitions and connections in statistics that need to be explained:

- Mean - μ , the average of a random variable. An easy way to calculate μ iteratively is the following. Since

$$\hat{\mu}_n = \frac{1}{n} \sum_{k=1}^n x_k = \frac{n-1}{n} \cdot \frac{1}{n-1} \sum_{k=1}^{n-1} x_k + \frac{1}{n} x_n$$

we can express $\hat{\mu}_n$ in terms of the previous mean estimate $\hat{\mu}_{n-1}$:

$$\hat{\mu}_n = \frac{n-1}{n} \hat{\mu}_{n-1} + \frac{1}{n} x_n$$

Rearranging, we obtain a more intuitive update equation:

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n} (x_n - \hat{\mu}_{n-1})$$

- Variance - σ^2 , the variance of a random variable is a measure of the average squared distance of each sample from the mean of the variable. An easy way

to estimate σ^2 using the unbiased estimator (with the $1/(N - 1)$ factor) is the following. We can define M_2 as the sum of squared distances of each sample from the mean:

$$M_2 = \sum_{k=1}^N (x_k - \hat{\mu}_N)^2$$

Since we want to compute M_2 recursively, we use the following update formula:

$$M_2(N) = M_2(N - 1) + (x_N - \hat{\mu}_{N-1})(x_N - \hat{\mu}_N)$$

where $\hat{\mu}_N$ is the updated mean. Finally, we obtain the variance estimate as:

$$\sigma^2 = \frac{M_2}{N - 1}$$

4.2.2 Alpha Filter

If a process is changing slowly, we estimate its next state by converging to its approximate mean based on recent samples. In such cases, there is a trade-off between accurately computing the mean and maintaining a fast update rate. A filter designed to address this need is the α filter, where $0 < \alpha < 1$ is a parameter that controls the balance between smoothing and responsiveness. The formula for this filter is:

$$y_n = y_{n-1} + \alpha(x_n - y_{n-1})$$

- By applying the Fourier transform, we can see that this filter behaves as a low-pass filter (LPF) with a DC gain of 1.
- The bandwidth (BW) of the filter depends on α : - If α is small, the BW is short and the filter averages over a longer history, leading to stronger smoothing but slower responsiveness. - If α is large, the BW is larger and filter reacts more quickly to changes but provides less smoothing.

4.2.3 Alpha-Beta Filter

In cases where the process is quasi-linear (for example, a slowly changing velocity), the smoothing provided by the α filter might be too aggressive, as it does not account for changes in velocity. In such scenarios, a better approach is the α - β filter, which extends the α filter by including a β parameter to also smooth velocity estimates. Consider the example of an object moving at a constant velocity. In this case, the relationship between position and velocity is given by:

$$y_{n+1} = y_n + \Delta t \cdot \dot{y}_n$$

To approximate the velocity at time $n + 1$, we can use a similar approach to the α filter and "smooth" the velocity using the β parameter as follows:

$$\dot{y}_{n+1} = \dot{y}_n + \frac{\beta(x_{n+1} - y_n)}{\Delta t}$$

As for the position, we estimate it by combining the predicted position (based on the system's dynamics) with the measured position, using the α parameter:

$$y_{n+1} = y_n + \Delta t \cdot \dot{y}_n + \alpha(x_{n+1} - y_n)$$

- A higher value of β makes the filter respond more quickly to velocity changes, but results in less smooth estimates. Similarly, a higher value of α makes the filter rely more on the measurements and less on the model prediction, and vice versa.
- In a similar way, we can construct filters that take into account higher-order derivatives of the system dynamics.

4.2.4 Noise Types Definition

In order to analyze systems, we should first define and distinguish between several types of noise:

- **Measurement noise** – When measuring process states, we use instruments that are not perfect. The random errors introduced by these instruments are referred to as measurement noise.
- **Process noise** – Many processes inherently include random elements, which are modeled as part of the system’s dynamics. These uncertainties are referred to as process noise.
- **Systematic measurement error**- discusses the quality of the measurement. There are cases in which our method of measurement is not perfect (for example, sometimes measurements contains time delay that might lead to miscalculations) In the α - β filter, we choose parameters accordingly to account for this error. The Kalman filter relies on an unbiased system.

4.3 Kalman Filter in 1D

4.3.1 Background on the Kalman Filter

The Kalman filter is a type of filter used to track noisy motion. Its concept is similar to the α - β filter; however, in the case of the Kalman filter, the PDF (probability distribution function) of the measurement and the estimation relied on normal distribution and the corresponding parameters are adaptively updated based on the measurement variance in order to minimize the mean squared error (MSE). The operation of the Kalman filter can be described in the following stages:

- **Initialization** – Initially, we provide the filter with an initial estimate of the system state and its associated uncertainty (variance).
- **Measurement** – At each iteration, a new measurement of the system state is taken, along with an estimate of the measurement variance.
- **Update** – Using the new measurement and the previous estimates of the system state, the filter updates its estimation of the current state and dynamics (e.g., position and velocity).
- **Prediction** – Based on the current state estimate and knowledge of the system’s dynamics, the filter predicts the state for the next iteration. This prediction is

then refined in the next cycle using new measurements, similar in spirit to the α - β filter.

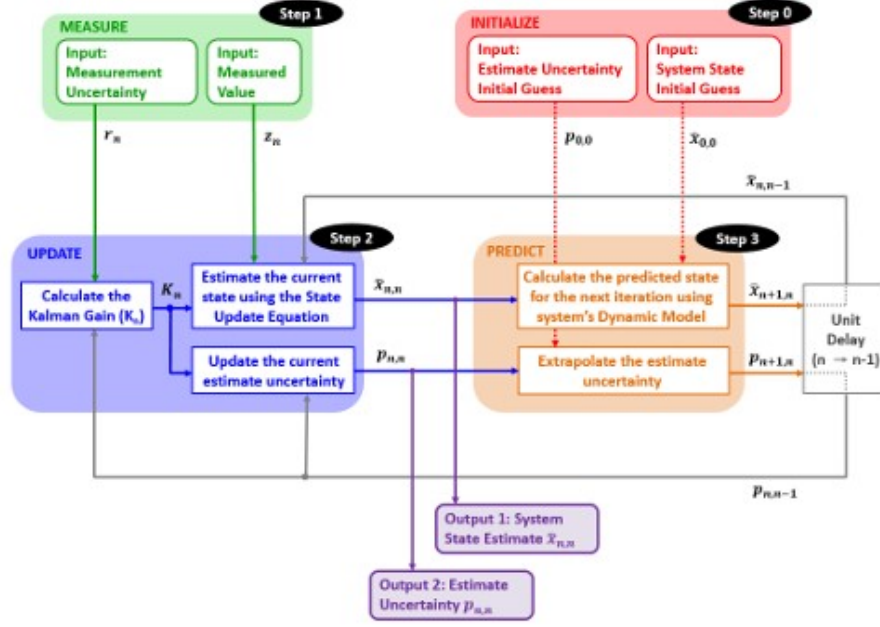


Figure 4.1: Kalman filter stages scheam

4.3.2 Kalman Filter for a Constant Variable

Assuming the system dynamics are zero, we can describe it using an α filter. To optimize the α parameter, we aim to minimize the variance of our estimated states. According to the α filter ($x_{n+1|n}$ stands for the approximation for x_{n+1} t time n):

$$x_n = \alpha \cdot z_n + (1 - \alpha)\hat{x}_{n-1}$$

The variance of our estimated states can be described with the following connection, where r_n is the uncertainty of our measurements (A constant number defined by out measuring tools and method):

$$p_n = \alpha^2 \cdot r_n + (1 - \alpha)^2 p_{n-1}$$

To minimize this expression, we differentiate it with respect to α and find the minimum. This results in the optimal value of α , which corresponds to the Kalman gain:

$$K_n = \frac{p_{n-1}}{p_{n-1} + r_n}$$

Using this gain, the variance can be updated as follows:

$$p_n = (1 - K_n) \cdot p_{n-1}$$

In the case of a constant process, no prediction step is required. The role of the Kalman gain is similar to the α parameter in an α filter. When the measurements variance is high, K_n approaches 0 and the filter relies more on the previous estimate ignoring new measurements. Conversely, when the measured variance is low, K_n approaches 1 and the filter gives more weight to the new measurement, assuming low noise.

4.3.3 Five Kalman Filter Equations

In general, to build a Kalman filter, we need to define five equations that describe its operation. The equations for a constant random variable are given as an example, where x represents the estimated state, z represents the measurements, p is the estimated covariance of the state uncertainty, r is the measurement noise variance, q is the process noise variance, and K is the Kalman gain, which determines how much to trust the measurement versus the prediction. The five Kalman filter equations are:

- **State update** – Updating the estimation for the system's state ($x_{n+1|n}$ stands for the approximation for x_{n+1} at time n):

$$x_n = x_{n-1} + K_n(z_n - x_{n|n-1})$$

- **Covariance update** – Updating our estimate of the covariance:

$$p_n = (1 - K_n)p_{n-1}$$

- **Kalman gain update** – Updating the Kalman gain according to the estimated and measured variance:

$$K_n = \frac{p_{n-1}}{p_{n-1} + r_n}$$

- **State prediction** – Predicting the next system state based on knowledge of the system's dynamics:

$$\hat{x}_{n+1} = \hat{x}_n$$

- **Covariance prediction** – Predicting the covariance for the next iteration:

$$p_{n+1} = p_n + q_n$$

4.3.4 Kalman filter for changing variable

Now, suppose a variable that changes with approximately constant velocity. There are two differences between such case to the constant one:

- Prediction phase - now the next iteration state estimation is also take into account the system's dynamic:

$$\hat{x}_{n+1} = x_n + \Delta t \cdot \dot{x}_n$$

Hence, using measurements the estimated state is:

$$x_{n+1} = x_{n+1|n} + K_n \cdot (z_n - x_{n+1|n})$$

- Variance calculation - The relationship between the state and "velocity" variance can be shown as the following:

$$P_{n+1}^x = P_n^x + \Delta t^2 \cdot P_n^{\dot{x}}$$

While the velocity uncertainty stay the same:

$$P_{n+1}^{\dot{x}} = P_n^{\dot{x}}$$

Over all, the equations describing constant velocity Kalman filter are the one presented in the attached image:

	Equation	Equation Name	Alternative names used in the literature
State	$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n(z_n - \hat{x}_{n,n})$	State Update	Filtering Equation
Update	$p_{n,n} = (1 - K_n)p_{n,n-1}$	Covariance Update	Corrector Equation
	$K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n}$	Kalman Gain	Weight Equation
State	$\hat{x}_{n+1,n} = \hat{x}_{n,n}$ (for constant dynamics)	State Extrapolation	Predictor Equation
Predict	$\hat{x}_{n+1,n} = \hat{x}_{n,n} + \Delta t \hat{\dot{x}}_{n,n}$		Transition Equation
	$\hat{\dot{x}}_{n+1,n} = \hat{\dot{x}}_{n,n}$ (for constant velocity dynamics)		Prediction Equation Dynamic Model State Space Model
	$p_{n+1,n}^x = p_{n,n}^x$ (for constant dynamics)	Covariance Extrapolation	Predictor Covariance
	$p_{n+1,n}^x = p_{n,n}^x + \Delta t^2 p_{n,n}^v$		Equation
	$p_{n+1,n}^v = p_{n,n}^v$ (for constant velocity dynamics)		

Figure 4.2: Constant velocity kalman filter equations

4.3.5 Process Noise

When analyzing real systems, our model is usually not 100% accurate. For example, if we assume a car is moving at a constant speed, external factors like wind that change the car's speed are not included in our model and might degrade the performance of the filter's estimations. To handle this, we acknowledge that our model can never reach zero uncertainty. This can be modeled by adding a small constant to the estimation variance at each iteration. This constant represents the process noise and ensures that the filter remains adaptive to unexpected changes in the system. The update of the state uncertainty P due to process noise is usually modeled as:

$$P_{n+1} = P_n + Q$$

where Q is the process noise addition, typically chosen as a small positive constant value.

4.3.6 Lag error

Lag error refers to the difference between the true state values and the estimated values. This discrepancy often arises when an important system parameter—such as acceleration in the α - β filter—is neglected. One way to mitigate this error is by improving the model, for example, by introducing an additional gain to account for the missing dynamics. Another way is to adjust the reliability of the process model by increasing the process noise. Thus, we will update the Covariance Extrapolation. The first approach generally yields more accurate estimations, as it better represents the actual system behavior. However, having an accurate model is not always feasible in practice. As a result, the second method—adjusting the process noise—is more commonly utilized.

4.3.7 Confidence interval

Is a statistical range, calculated from the estimated parameters, within which the true value of an unknown parameter is expected to lie with a certain probability. For example, for a normal distribution.

$$P(-1.96 \leq Z \leq 1.96) \approx 0.95$$

$$\hat{x} \pm 1.96 \cdot \sigma$$

By the expression above, we ensure that there is a 95% chance that the true value lies within the delta around our estimate. The choose of 95% is useful since it providing a high level of confidence in finding the hidden state. The lower our measurement uncertainty, the narrower our "sleeve" (confidence interval) becomes.



Figure 4.3: High Sigma

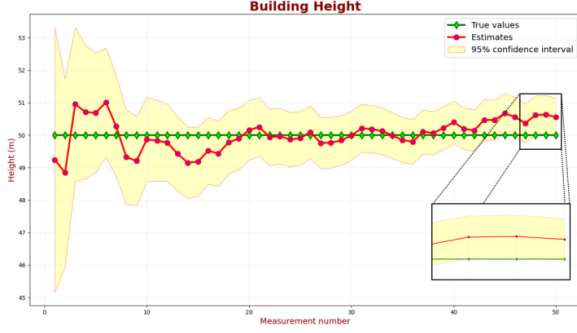


Figure 4.4: Low Sigma

4.4 Kalman filter in N-D

4.4.1 Formalization of High-Dimensional Kalman Filter

To handle multiple dimensions, we describe the system using a state vector that includes position, velocity, and possibly higher-order derivatives (e.g., acceleration, jerk), such as:

$$\mathbf{x} = (x, y, z, \dot{x}, \dot{y}, \dot{z}, \ddot{x}, \ddot{y}, \ddot{z}, \dots)^T$$

In this high-dimensional setting, the Kalman filter operates with matrix-valued quantities:

- $\mathbf{P}_{n|n}$ – the **a posteriori** (estimation) error covariance matrix
- $\mathbf{P}_{n+1|n}$ – the **a priori** (prediction) error covariance matrix
- $\mathbf{R}_n$ – the measurement noise covariance matrix
- $\mathbf{Q}$ – the process noise covariance matrix

The output of the Kalman filter at each step is a high-dimensional Gaussian distribution representing the belief over the system state. This is expressed as:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^N |\mathbf{P}|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{P}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

where:

- $\mathbf{x}$ is the state vector
- $\boldsymbol{\mu}$ is the mean (typically the current estimate)
- $\mathbf{P}$ is the state covariance matrix
- N is the dimensionality of the state vector

4.4.2 State prediction Equation

The state update equation for a high-dimensional Kalman filter is given by:

$$\hat{\mathbf{x}}_{n+1|n} = \mathbf{F}\hat{\mathbf{x}}_{n|n} + \mathbf{G}\mathbf{u}_n + \mathbf{w}_n$$

where:

- $\hat{\mathbf{x}}_{n+1|n}$ is the predicted state at time $n + 1$ given information up to time n ,
- $\hat{\mathbf{x}}_{n|n}$ is the current estimated state,
- $\mathbf{F}$ is the state transition matrix,
- $\mathbf{G}$ is the control input matrix, determining how the control input $\mathbf{u}_n$ influences the state evolution,
- $\mathbf{w}_n$ is the process noise vector.

As an example, suppose the state vector represents 2D position, velocity, and acceleration:

$$\mathbf{x} = (x, y, \dot{x}, \dot{y}, \ddot{x}, \ddot{y})^T$$

Then, assuming constant acceleration and uniform time steps of size Δt , the state transition matrix $\mathbf{F}$ would be:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 1 & 0 & \Delta t & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix models the kinematic relationships between position, velocity, and acceleration under Newtonian motion. As for the control input $\mathbf{u}_n$ and the matrix $\mathbf{G}$, the vector $\mathbf{u}_n$ may represent known external inputs applied to the system. For example, it could describe a controlled change in acceleration (such as thrust or force). In that case, the matrix $\mathbf{G}$ encodes how these inputs affect the state evolution — similarly to how $\mathbf{F}$ models the internal kinematic relationships. Because acceleration affects both velocity and position over time, $\mathbf{G}$ would reflect these connections. For instance, if $\mathbf{u}_n$

contains 2D acceleration commands, a suitable form for $\mathbf{G}$ (assuming constant input over time step Δt) could be:

$$\mathbf{G} = \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 \\ 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 \\ 0 & \Delta t \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

This matrix maps the control acceleration inputs to changes in position, velocity, and acceleration in both the x - and y -directions.

4.4.3 Covariance prediction Equation

The covariance update equation for a high-dimensional Kalman filter is given by:

$$\mathbf{P}_{n+1|n} = F\mathbf{P}_{n|n}F^T + Q$$

where:

- $\mathbf{P}_{n+1|n}$ is the uncertainty of the predictions and $\mathbf{P}_{n|n}$ is the uncertainty of our estimations
- Q is the process noise matrix

The development of this equation can be easily made by using the covariance matrix definition and state update equation.

4.4.4 State Update Equation

Even in the high-dimensional case, we want to estimate the current state by combining prior predictions with current measurements. This is accomplished using the Kalman gain, which determines how much weight we give to the new measurement relative to our prior estimate. The state correction equation is:

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + \mathbf{K}_n (\mathbf{Z}_n - \mathbf{H}\hat{\mathbf{x}}_{n|n-1})$$

where:

- $\hat{\mathbf{x}}_{n|n}$ is the updated (posterior) state estimate,
- $\hat{\mathbf{x}}_{n|n-1}$ is the predicted (prior) state estimate,
- $\mathbf{Z}_n$ is the current measurement vector,
- $\mathbf{H}$ is the observation matrix that maps the state space to the measurement space. If we have direct access to all components of the state vector through measurements, then $\mathbf{H}$ is the identity matrix.
- $\mathbf{K}_n$ is the Kalman gain matrix.

The Kalman gain in the high-dimensional case is computed analogously to the 1D case:

$$\mathbf{K}_n = \mathbf{P}_{n|n-1} \mathbf{H}^T (\mathbf{H} \mathbf{P}_{n|n-1} \mathbf{H}^T + \mathbf{R}_n)^{-1}$$

where:

- $\mathbf{P}_{n|n-1}$ is the prior error covariance matrix,
- $\mathbf{R}_n$ is the measurement noise covariance matrix.

This gain matrix ensures that the updated estimate $\hat{\mathbf{x}}_{n|n}$ appropriately balances the uncertainty in the prediction and the measurement.

4.4.5 Covariance Update Equation

The state covariance matrix, which quantifies the uncertainty in the state estimate, is also updated after incorporating the new measurement. The covariance update equation is:

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n \mathbf{H}) \mathbf{P}_{n|n-1} (\mathbf{I} - \mathbf{K}_n \mathbf{H})^T + \mathbf{K}_n \mathbf{R}_n \mathbf{K}_n^T$$

where:

- $\mathbf{P}_{n|n}$ is the updated (posterior) error covariance matrix,
- $\mathbf{P}_{n|n-1}$ is the predicted (prior) error covariance matrix,
- $\mathbf{K}_n$ is the Kalman gain matrix,
- $\mathbf{H}$ is the observation matrix,
- $\mathbf{R}_n$ is the measurement noise covariance matrix,
- $\mathbf{I}$ is the identity matrix of appropriate dimension.

This form of the update ensures that uncertainty is reduced appropriately based on the relative confidence in the prediction and the measurement. It also helps maintain numerical stability in high-dimensional filters. An alternative and often used simplification (especially in implementations) is:

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n \mathbf{H}) \mathbf{P}_{n|n-1}$$

This version is algebraically valid under the assumption of optimal Kalman gain and certain noise properties, but may be less robust in the presence of numerical errors or imperfect models.

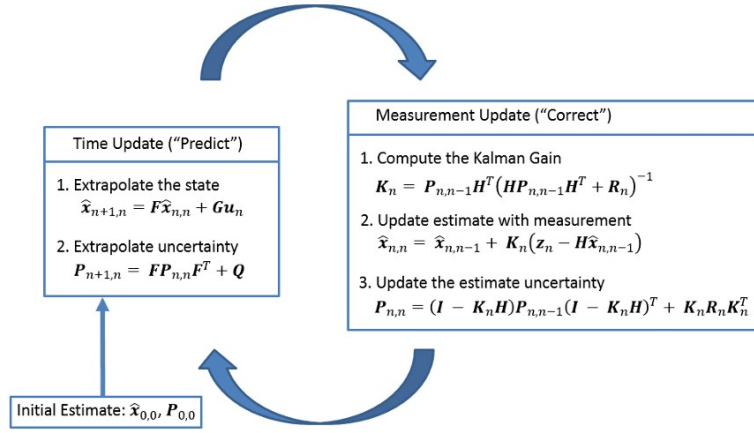


Figure 8.8: The Kalman Filter Diagram.

Figure 4.5: High dimensional kalman filter five equations

4.5 What is a non linear system

Up to this point, our discussion has focused on linear systems. In practice, however, many real-world problems exhibit nonlinear behavior. It is therefore important to identify the conditions under which a system transitions from linear to nonlinear dynamics. In this context, we distinguish between two main types of nonlinearities:

- **State-to-measurement nonlinearity:** The observation matrix H cannot be defined in a meaningful way due to a nonlinear relationship between the state variables and the measurements.
- **Nonlinear system dynamics:** In a linear system, the relationship between the current state and the next state can be expressed as a simple weighted sum, much like in linear regression, where outputs are a linear combination of inputs. This property allows the state-transition model to be represented by a constant matrix F . However, when the relationship cannot be fully captured by a fixed set of linear weights, F cannot be defined in a meaningful way.

Moreover, the measurement and estimation errors may no longer follow a Gaussian distribution, which violates one of the core assumptions of the Kalman Filter.

To address this issue, two main approaches can be used:

- **Extended Kalman Filter (EKF)** – performs analytical linearization of the model around the current estimate
- **Unscented Kalman Filter (UKF)** – performs statistical linearization using the unscented transform

Although the UKF generally provides better accuracy than the EKF with comparable computational complexity, we will use the EKF for simplicity in this document.

We will investigate the oscillatory motion of an object, with the goal of estimating the angle of arrival based on observations of previous angle measurements.

4.6 EKF – Extended Kalman Filter

4.6.1 Linear Approximation

The Extended Kalman Filter (EKF) is an extension of the standard (linear) Kalman filter that allows it to handle nonlinear systems. It does so by approximating the nonlinear system dynamics and measurement models using a first-order Taylor expansion, i.e., by linearizing them around the current state estimate. The first-order Taylor approximation of a nonlinear function $f(x)$ around a point x_0 is given by:

$$f(x) \approx f(x_0) + \dot{f}(x_0)(x - x_0)$$

where $\dot{f}(x_0)$ is the derivative (or the Jacobian matrix in the multivariate case) of f , evaluated at x_0 . When dealing with Gaussian random variables, this linear approximation enables us to estimate how uncertainty propagates through a nonlinear function. If the input variable has a standard deviation σ_x , then the output standard deviation σ_f is:

$$\sigma_f = \dot{f}(x)\sigma_x$$

and the variance propagates as:

$$P_f = \dot{f}(x)^2 P_x$$

In the EKF, as in the Linear Kalman Filter (LKF), both the state estimate and the error covariance matrix evolve over time via matrix multiplications which are linear operations. However, unlike the LKF which uses fixed system matrices, the EKF uses matrices that are recalculated at every time step based on the current state estimate. This means the EKF can adapt to local nonlinear behavior by updating its linear approximation continuously, allowing it to track states in nonlinear systems more effectively than the LKF.

4.6.2 Observation Matrix

In nonlinear systems, the measurements are often not direct observations of the state variables but rather nonlinear functions of them. For example, when tracking an object's angle relative to a fixed point, the measurement may be an angle θ that depends on the position x , such as:

$$z = \theta = h(x) = \arctan\left(\frac{x}{d}\right)$$

In such cases, we must linearize the observation function $h(x)$ to apply the EKF update step. This is done by computing the Jacobian (in one dimension, the derivative) of $h(x)$ with respect to x :

$$H = \frac{\partial h(x)}{\partial x} = \frac{d}{d^2 + x^2}$$

This Jacobian becomes the observation matrix H in the EKF update equations, and it is recalculated at each time step based on the current estimate of x . In the general case, where the observation function is a nonlinear vector-valued function $h(\mathbf{x})$, the

observation matrix is obtained by computing the Jacobian of h with respect to the state vector $\mathbf{x}$:

$$H = \left. \frac{\partial h(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}} = \begin{bmatrix} \frac{\partial h_0}{\partial x_0} & \frac{\partial h_0}{\partial x_1} & \cdots & \frac{\partial h_0}{\partial x_{n-1}} \\ \frac{\partial h_1}{\partial x_0} & \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_{n-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_{m-1}}{\partial x_0} & \frac{\partial h_{m-1}}{\partial x_1} & \cdots & \frac{\partial h_{m-1}}{\partial x_{n-1}} \end{bmatrix}$$

Here, $\hat{\mathbf{x}}$ is the current state estimate, and H is evaluated at that point. This linearized observation matrix allows the EKF to apply the standard Kalman filter update equations even in the presence of nonlinear measurement models.

4.6.3 Transformation Matrix

As for the dynamic model of the system, it can also be described using linearization by turning the state transition matrix F into a function of the current state $\mathbf{x}$. In a nonlinear system, the state evolves according to a nonlinear function:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1})$$

The transition matrix F can be described as the local linearization jacobian of the function f :

$$F_k = \left. \frac{\partial f(\mathbf{x}, \mathbf{u})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{k-1}, \mathbf{u}=\mathbf{u}_{k-1}} = \begin{bmatrix} \frac{\partial f_0}{\partial x_0} & \frac{\partial f_0}{\partial x_1} & \cdots & \frac{\partial f_0}{\partial x_{n-1}} \\ \frac{\partial f_1}{\partial x_0} & \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_{n-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_{n-1}}{\partial x_0} & \frac{\partial f_{n-1}}{\partial x_1} & \cdots & \frac{\partial f_{n-1}}{\partial x_{n-1}} \end{bmatrix}$$

	Equation	LKF Equation	EKF Equation
Predict	State	$\hat{\mathbf{x}}_{n+1,n} = \mathbf{F} \hat{\mathbf{x}}_{n,n} + \mathbf{G} \mathbf{u}_n$	$\hat{\mathbf{x}}_{n+1,n} = \mathbf{f}(\hat{\mathbf{x}}_{n,n}) + \mathbf{G} \mathbf{u}_n$
	Extrapolation		
	Covariance	$\mathbf{P}_{n+1,n} = \mathbf{F} \mathbf{P}_{n,n} \mathbf{F}^T + \mathbf{Q}$	$\mathbf{P}_{n+1,n} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \mathbf{P}_{n,n} \left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right)^T + \mathbf{Q}$
Update	State Update	$\hat{\mathbf{x}}_{n,n} = \hat{\mathbf{x}}_{n,n-1} + \mathbf{K}_n (\mathbf{z}_n - \mathbf{H} \hat{\mathbf{x}}_{n,n-1})$	$\hat{\mathbf{x}}_{n,n} = \hat{\mathbf{x}}_{n,n-1} + \mathbf{K}_n (\mathbf{z}_n - \mathbf{h}(\hat{\mathbf{x}}_{n,n-1}))$
	Covariance	$\mathbf{P}_{n,n} = (\mathbf{I} - \mathbf{K}_n \mathbf{H}) \mathbf{P}_{n,n-1} \times (\mathbf{I} - \mathbf{K}_n \mathbf{H})^T + \mathbf{K}_n \mathbf{R}_n \mathbf{K}_n^T$	$\mathbf{P}_{n,n} = \left(\mathbf{I} - \mathbf{K}_n \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right) \mathbf{P}_{n,n-1} \times \left(\mathbf{I} - \mathbf{K}_n \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right)^T + \mathbf{K}_n \mathbf{R}_n \mathbf{K}_n^T$
	Kalman Gain	$\mathbf{K}_n = \mathbf{P}_{n,n-1} \mathbf{H}^T \times (\mathbf{H} \mathbf{P}_{n,n-1} \mathbf{H}^T + \mathbf{R}_n)^{-1}$	$\mathbf{K}_n = \mathbf{P}_{n,n-1} \left(\frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right)^T \times \left(\frac{\partial \mathbf{h}}{\partial \mathbf{x}} \mathbf{P}_{n,n-1} \left(\frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right)^T + \mathbf{R}_n \right)^{-1}$

Table 13.1: Kalman Filter equations.

Figure 4.6: EKF equations

4.6.4 LKF for Oscillator

Consider a simple harmonic oscillator, where the state vector is defined as:

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}$$

Here, θ is the angle and $\dot{\theta}$ is the angular velocity.

The continuous-time dynamics of the system are governed by the second-order differential equation:

$$\ddot{\theta} = -\omega^2 \theta$$

Since the system is linear, we can discretize it with sampling time Δt and represent the state update using the state transition matrix F :

$$F = \begin{bmatrix} 1 & \Delta t \\ -\Delta t \omega^2 & 1 \end{bmatrix}.$$

The observation matrix H is straightforward since we directly estimate θ from its own measurement:

$$H = [1 \quad 0]$$

4.6.5 EKF for Oscillator

Consider a simple harmonic oscillator, where the state vector is defined as:

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \\ \omega \end{bmatrix}$$

In this system the non-linearity comes from non-linear system dynamics. Thus, we can not express our system with matrix F . The system dynamics $\mathbf{f}(\mathbf{x}_n)$ are:

$$\mathbf{f}(\mathbf{x}_n) = \begin{bmatrix} \theta_{n-1} + \Delta t \dot{\theta}_{n-1} \\ \dot{\theta}_{n-1} - \omega_{n-1}^2 \theta_{n-1} \Delta t \\ \omega_{n-1} \end{bmatrix} \quad (4.1)$$

To linearize the system around the current state estimate, we compute the Jacobian of $f(\mathbf{x})$, which will preserve the Gaussian distribution of the uncertainty matrix:

$$F_J = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \theta_{n+1}}{\partial \theta_n} & \frac{\partial \theta_{n+1}}{\partial \dot{\theta}_n} & \frac{\partial \theta_{n+1}}{\partial \omega_n} \\ \frac{\partial \dot{\theta}_{n+1}}{\partial \theta_n} & \frac{\partial \dot{\theta}_{n+1}}{\partial \dot{\theta}_n} & \frac{\partial \dot{\theta}_{n+1}}{\partial \omega_n} \\ \frac{\partial \omega_{n+1}}{\partial \theta_n} & \frac{\partial \omega_{n+1}}{\partial \dot{\theta}_n} & \frac{\partial \omega_{n+1}}{\partial \omega_n} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & 0 \\ -\omega_k^2 \Delta t & 1 & -2\Delta t \cdot \omega_k \cdot \theta_k \\ 0 & 0 & 1 \end{bmatrix} \quad (4.2)$$

The observation matrix H is straightforward since we directly estimate θ from its own measurement. Therefore, there is no need to compute its Jacobian:

$$H = [1 \quad 0 \quad 0] \quad (4.3)$$

4.7 Chapter Summary

Since the angular velocity ω in our system varies with wind changes, the system exhibits nonlinear behavior. Consequently, we model the column movement according to equations (4.1), (4.2), and (4.3).

#Kalman Filter

Linear Kalman Filter (LKF)

```
class LKF:
    def __init__(self, init_pos, init_vel, init_var, measurments_var,
process_variance, omega = 0):
        self.mX = np.array([[init_pos],
                             [init_vel]])
        self.mP = np.array([[init_var, 0],
                             [0, init_var]])
        self.mR = np.array([[measurments_var]])
        self.mQ = process_variance * np.eye(2)
        self.mOmega = omega

    def GetSampels(self, input, dt):
        if isinstance(input, np.ndarray):
            outVec = []
            outVarVec = []
            for sample in input:
                if -91 < sample < 91:
                    pos, var = self.ApplyFilter(sample, dt)
                else:
                    pos, var = self.Override(dt)
                outVec.append(pos)
                outVarVec.append(var)
            return np.array(outVec), np.array(outVarVec)
        else:
            if -91 < input < 91:
                pos, var = self.ApplyFilter(input, dt)
            else:
                pos, var = self.Override(dt)
            return pos, var

    def Override(self, dt):
        F = np.array([[1, dt],
                      [-self.mOmega**2 * dt, 1]])
        H = np.array([[1, 0]])
        I = np.eye(2)

        # --- Prediction only ---
        self.mX = F @ self.mX
        self.mP = F @ self.mP @ F.T + self.mQ

        return self.mX[0, 0], self.mP[0, 0]

    def ApplyFilter(self, measuredPos, dt):
        F = np.array([[1, dt],
                      [-self.mOmega**2 * dt, 1]])
        H = np.array([[1, 0]])
```

```

I = np.eye(2)

# --- Prediction ---
self.mX = F @ self.mX
self.mP = F @ self.mP @ F.T + self.mQ

# --- Update ---
z = np.array([[measuredPos]])
y = z - H @ self.mX
S = H @ self.mP @ H.T + self.mR
K = self.mP @ H.T @ np.linalg.inv(S)

# --- Estimation ---
self.mX = self.mX + K @ y
self.mP = (I - K @ H) @ self.mP

return self.mX[0, 0], self.mP[0, 0]

```

Extended Kalman Filter (EKF)

For oscillator tracking

```

class EKF:
    def __init__(self, init_angle, init_angularVelocity, init_Omega,
init_var, measurments_var, proccess_variance):
        self.mX = np.array([[init_angle],
                             [init_angularVelocity],
                             [init_Omega]])
        self.mP = init_var * np.eye(3)
        self.mR = np.array([[measurments_var]])
        self.mQ = proccess_variance * np.eye(3)
        self.mH = np.array([[1,0,0]])
        self.mI= np.eye(3)

    def GetSampels(self, input, dt):
        if isinstance(input, np.ndarray):
            outVec = []
            outVarVec = []
            for sample in input:
                if -91 < sample < 91:
                    pos, var = self.ApplyFilter(sample, dt)
                else:
                    pos, var = self.Override(dt)
                outVec.append(pos)
                outVarVec.append(var)
            return np.array(outVec), np.array(outVarVec)
        else:
            if -91 < input < 91:
                pos, var = self.ApplyFilter(input, dt)

```



```

        else:
            pos, var = self.Override(dt)
            return pos, var

    def Override(self, dt):
        new_theat= self.mX[0,0]+ dt*self.mX[1,0]
        new_theta_dot= self.mX[1,0] - dt*self.mX[0,0]*self.mX[2,0] **
2
        new_omega = self.mX[2,0]

        F_Jacobian = np.array([[1, dt,0],
                                [-(self.mX[2,0]**2) * dt, 1, -2*
self.mX[2,0] * self.mX[0,0] * dt],
                                [0, 0, 1]])

        # --- Prediction only ---
        self.mX = np.array([[new_theat],
                             [new_theta_dot],
                             [new_omega]])
        self.mP = F_Jacobian @ self.mP @ F_Jacobian.T + self.mQ

        return self.mX[0, 0], self.mP[0, 0]

    def ApplyFilter(self, measuredPos, dt):
        #calculate f(x_(n,n)) because it is not linear and we can not
write it as a matrix
        new_theat= self.mX[0,0]+ dt*self.mX[1,0]
        new_theta_dot= self.mX[1,0] - dt*self.mX[0,0]*self.mX[2,0] **
2
        new_omega = self.mX[2,0]

        F_Jacobian = np.array([[1, dt,0],
                                [-(self.mX[2,0]**2) * dt, 1, -2*
self.mX[2,0] * self.mX[0,0] * dt],
                                [0, 0, 1]])

        # --- Prediction ---
        self.mX = np.array([[new_theat],
                             [new_theta_dot],
                             [new_omega]])
        self.mP = F_Jacobian @ self.mP @ F_Jacobian.T + self.mQ

        #-----Kalman Update-----
        K= self.mP @ self.mH.T @ np.linalg.inv( self.mH @ self.mP @
self.mH.T+ self.mR)

        # --- Update ---
        z = np.array([[measuredPos]])

```

```
self.mX= self.mX + K @ (z-self.mH@ self.mX)
self.mP = (self.mI - K @ self.mH) @ self.mP

return self.mX[0, 0], self.mP[0, 0]
```

Chapter 5

Phased and Gain distortion

5.1 introduction

As frequency increases, issues that were negligible at low frequencies can no longer be ignored. In this chapter, we introduce and propose solutions to the coupling problem. We have based our work in this chapter on [4]

5.2 Coupling

5.2.1 IQ imbalance

IQ imbalance (IQI) in direct-conversion receivers (DCR) refers to mismatches in amplitude and phase between the in-phase (I) and quadrature (Q) signal paths. These mismatches can arise from antenna coupling and environmental coupling. Mathematically, the received signal can be modeled as:

$$x(t) = z_m^I(t) + jz_m^Q(t) * h_m(t) + n_m(t) \quad (5.1)$$

where:

- $z_m^I(t)$ is the in-phase component of the transmitted signal,
- $z_m^Q(t)$ is the quadrature component,
- $h_m(t)$ models the IQ imbalance effect (impulse response),
- $n_m(t)$ is the additive noise.

5.2.2 Couplings model

In a uniform linear array (ULA) with equal spacing between elements, the mutual coupling is often modeled using a Mutual Coupling Matrix (MCM), typically represented as a Toeplitz matrix. However, this model does not account for environmental coupling effects, which can significantly impact array performance in practical scenarios.

There exists a correlation between gain-phase (GP) mismatch and the *coupling matrix* that we aim to estimate. Therefore, we model the received signal as:

$$z(t) = D(\theta_d)a(\theta_d)s_d(t) + D(\theta_i)a(\theta_i)s_i(t) \quad (5.2)$$

where:

- θ_d is the direction of the desired signal,
- θ_i is the direction of the interfering signal,
- $a(\theta)$ is the array steering vector,
- $s_d(t)$ and $s_i(t)$ are the source signals (desired and interference, respectively),
- $D(\theta)$ is a diagonal matrix modeling the gain and phase mismatches.

The mismatch matrix $D(\theta)$ is defined as:

$$D(\theta) = \text{diag} \left((1 + g_0(\theta))e^{j\phi_0(\theta)}, \dots, (1 + g_{M-1}(\theta))e^{j\phi_{M-1}(\theta)} \right) \quad (5.3)$$

where:

- $g_m(\theta)$ represents the gain mismatch of the m -th element,
- $\phi_m(\theta)$ represents the phase mismatch of the m -th element,
- M is the number of RF paths.

5.3 Minimizing the EVM (Error Vector Magnitude)

Our objective is to minimize the Error Vector Magnitude (EVM) by making the received signal $x(t)$ as close as possible to the ideal signal $a(\theta_d)s_d(t)$, where θ_d is the direction of the desired source.

To achieve this, we adopt a two-stage correction strategy:

1. **IQ Imbalance Compensation:** We first mitigate the impact of the IQ imbalance by estimating and reducing $h_m(t)$ using a finite impulse response (FIR) filter. This filter is designed to compensate for amplitude and phase mismatches in the analog front-end.
2. **Coupling Matrix Calibration:** After IQI correction, we address the effect of the mismatch matrix $D(\theta)$ in the digital domain through calibration. This step aims to suppress the residual distortion caused by element-wise gain and phase errors and to better align the received signal with the steering vector $a(\theta_d)$.

Together, these steps aim to enhance signal fidelity and reduce the overall EVM. Since we operate in the digital domain, we will concentrate on implementing the second stage.

5.3.1 Reducing the GP mismatch

after the IQI correction we will calibrate the system for K angles. on each angle we will calculate the GP mismatch

Phase correction

We sample the output of the IQI-compensated signal after the FIR filter for N samples at an intermediate frequency $f = \text{IF}$, and perform a Discrete Fourier Transform (DFT) on the sampled data.

To isolate the phase distortion, we subtract the ideal steering vector $a(\theta_d)$ (evaluated at the desired direction) from the transformed signal. The resulting difference reflects the phase error.

$$\hat{\phi}_m(\theta_k^{\text{cal}}) = \angle \{ \text{DFT} \{ y_m(n, \theta_k^{\text{cal}}) \} (f = \text{IF}) \} - \phi_{a,m}(\theta) \quad (5.4)$$

We can represent it as a Matrix:

$$\hat{\Phi}(\Theta^{\text{cal}}) = [\hat{\phi}_m(\theta_1^{\text{cal}}), \dots, \hat{\phi}_m(\theta_k^{\text{cal}})]^T \quad (5.5)$$

Gain correction

In the same way, we can calculate the Gain mismatch:

$$1 + \hat{g}_m(\theta_k^{\text{cal}}) = \frac{\| \mathbf{y}_m(n, \theta_k^{\text{cal}}) \|_2}{\sqrt{N}} \quad (5.6)$$

$$\hat{\mathbf{G}}(\theta_k^{\text{cal}}) = [1 + \hat{g}(\theta_k^{\text{cal}}), \dots, 1 + \hat{g}(\theta_k^{\text{cal}})] \quad (5.7)$$

The steering vector outcome

With those mismatch retrieving we can calculate the correction by finding the inverse of it.

$$\tilde{\mathbf{a}}(\theta) = \mathbf{D}(\theta) \mathbf{a}(\theta) \quad (5.8)$$

,

$$\bar{\mathbf{D}}(\theta) = \text{diag}(\bar{\mathbf{G}}(\theta)) \text{diag} \left(e^{j\bar{\Phi}(\theta)} \right) \quad (5.9)$$

$$\hat{\mathbf{a}}(\theta) = \bar{\mathbf{D}}^{-1}(\theta) \tilde{\mathbf{a}}(\theta) \quad (5.10)$$

5.3.2 Chapter Summary

Since coupling cannot be neglected, the steering vector must be corrected as shown in 5.10. Consequently, the DOA algorithm must obtain the matrix D

phase and gain calculation

```
def
cphase(rawData,theta,wave_length,distance,num_of_antennas,reference_ant):
    """
    Calibrates phases using FFT phase estimation.

    Parameters:
    rawData (ndarray): n x 4 raw signal matrix (complex)
    reference_ant (int): Index of the reference antenna (0-based)

    Returns:
    fixed_rawData (ndarray): n x 4 phase-aligned signal matrix
    weights (ndarray): 4 x 1 vector of complex weights for phase
alignment
    """
    if not isinstance(rawData, np.ndarray) or not
isinstance(reference_ant, int):
        print("XxXxXxXxXxXxXxXxXx")
        print("rawData must be a matrix and reference_ant an integer")
        print("XxXxXxXxXxXxXxXxXx")
        exit()

    max_val = np.zeros(4, dtype=complex) # (0,0,0,0)

    # FFT and find the max in frequency domain
    for i in range(4):
        rawFt = np.fft.fftshift(np.fft.fft(rawData[:, i]))
        max_val[i] = rawFt[np.argmax(np.abs(rawFt))]

    angs = np.angle(max_val) #shape (4,) - 1 D
    ant_indices = np.arange(num_of_antennas) # (0,1,2,3) shape (4,) -
1 D
    steering_vector= (2*np.pi* ant_indices *
(distance/wave_length))*np.sin(np.deg2rad(theta))
    cal_angs= angs- steering_vector #shape (4,) - 1 D
    cal_angs= cal_angs- angs[reference_ant]
    weights = np.exp(-1j * cal_angs)
    fixed_rawData = rawData * weights # broadcasting applies each
weight column-wise
    return weights, fixed_rawData

def cgain(rawData, reference_ant):
    """
    Calibrates Gain using norm.

    Parameters:
    rawData (ndarray): n x 4 raw signal matrix (complex)
```

```

reference_ant (int): Index of the reference antenna (0-based)

Returns:
fixed_rawData (ndarray): n x 4 phase-aligned signal matrix
gain_mismatch (ndarray): 4 x 1 vector of gain weights
"""
if not isinstance(rawData, np.ndarray) or not
isinstance(reference_ant, int):
    print("XxXxXxXxXxXxXxXxXxXx")
    print("rawData must be a matrix and reference_ant an integer")
    print("XxXxXxXxXxXxXxXxXxXx")
    exit()

N = rawData.shape[0] # number of time samples
norm_val= np.zeros(4)
for i in range(4):
    antennaData=rawData[:,i]
    norm_val[i] = np.linalg.norm(antennaData) # Frobenius norm
(L2 over elements i)
#FIX- If theta is 0 return it and divide all with the first
antenna gain
gain_mismatch = (norm_val / np.sqrt(N))
gain_mismatch= gain_mismatch / gain_mismatch[reference_ant] #
FIX- maybe?
fixed_rawData= rawData * gain_mismatch
#gain_mismatch= np.zeros(4, dtype=complex)
return gain_mismatch, fixed_rawData

```

Chapter 6

Our System Model

6.0.1 System model

In order to account for pole movement while leveraging the benefits of the Kalman filter, we adopted the following system architecture:

- **Phased array** — The incoming signal is first collected using a fully digital phased array.
- **Signal corrections** — Two algorithms are applied to correct the received signal. The first is IQI (IQ imbalance) correction, and the second is GS (gain-phase) correction, which mitigates the effects of mutual coupling between elements.
- **MDL** — After producing the corrected signal, the MDL algorithm is used to estimate the number of angles of arrival (AoAs).
- **DOA algorithms** — Using the corrected signal and the estimated number of AoAs, we determine the incoming angles via DOA algorithms.
- **Kalman filter** — Finally, the noisy AoA estimates are processed by a Kalman filter to estimate the true pole position.

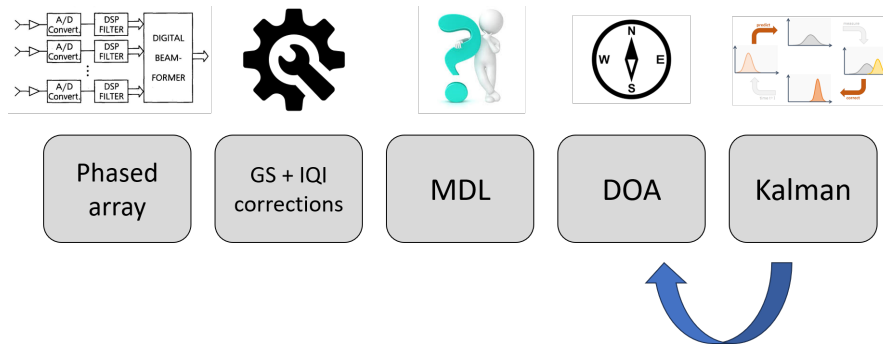


Figure 6.1: Final system architecture

To evaluate the system's performance, we tested it in two scenarios. First, we used a real 4-antenna digital phased array to assess the system's performance without any pole movement. Next, we built a simulation of a backhaul communication pole tracking another swinging backhaul pole, using the following parameters:

- **Antennas** — The received signal was assumed to come from a fully digital array with 20 antennas. The transmitting signal was directed toward our array, assuming that an initial calibration had been performed and that both the transmitter (TX) and receiver (RX) approximately knew each other's locations.
- **Pole movement** — The tracked pole oscillated at a frequency of 200 [Hz], located 100 [m] from the RX, with an oscillation amplitude of 0.5 [m].
- **Signals properties** — We used an RF signal with the frequency of 28 [GHz] and SNR of 10 [dB].

Chapter 7

Simulations

The simulations were carried out in the Colab environment using Python. The beam-forming section was inspired by [6]. The complete code is available on our GitHub repository: https://github.com/Danel-Var/Com_Project.

7.1 Real recordings analysis- Gain and Phase simulation

In our laboratory experiment, we investigated a single source emitting signals from various angles. The recorded signals were utilized to calibrate our system, and the performance was then compared with and without calibration.

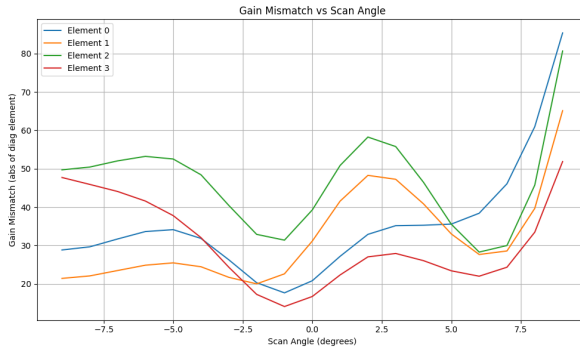


Figure 7.1: Gain mismatch

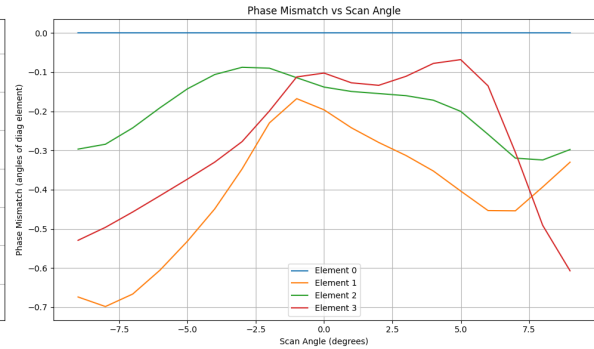


Figure 7.2: Phase mismatch

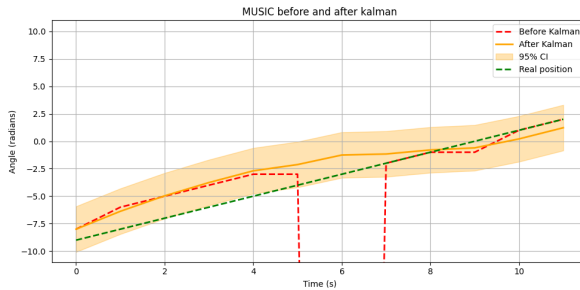


Figure 7.3: MUSIC Before Calibration

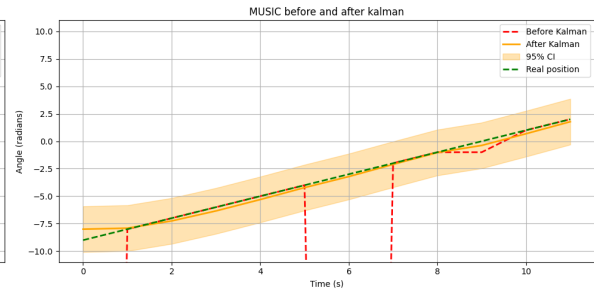


Figure 7.4: MUSIC After Calibration

From the results, we observe a clear improvement after calibration. It is important to note the drastic degradation in the “before calibration” case, where the signal appears noisy. In this scenario, the MDL algorithm fails to correctly estimate the number of received signals. Consequently, the Kalman filter disregards these unreliable measurements and instead relies on previously available information.

7.2 Swinging pole simulation

Using the parameters above, we observed two key points:

- **DOA algorithms** — At an SNR of 10 [dB], the performance of all the DOA algorithms we tested was similar. We believe that when a real system is used under different SNR conditions, and under several signals receiving with varying correlation level this result will change.
- **Estimation ratio** — For the EKF to reliably estimate the oscillation frequency of the pole, a sufficiently high DOA estimation rate is required. This makes sense because the EKF is based on linearization to handle the non-linear behavior of the system. Such an approximation is valid only when the system dynamics are slow enough, which is achieved at high estimation rates but not at low ones.

In Figure 7.5, it can be seen that for the three DOA algorithms used, the MSE of the position estimates relative to the true position of the pole is nearly identical. As expected, there is a minimal sampling ratio (corresponding to 10 [dB] SNR), beyond which increasing the estimation ratio leads to a monotonic improvement in estimation quality.

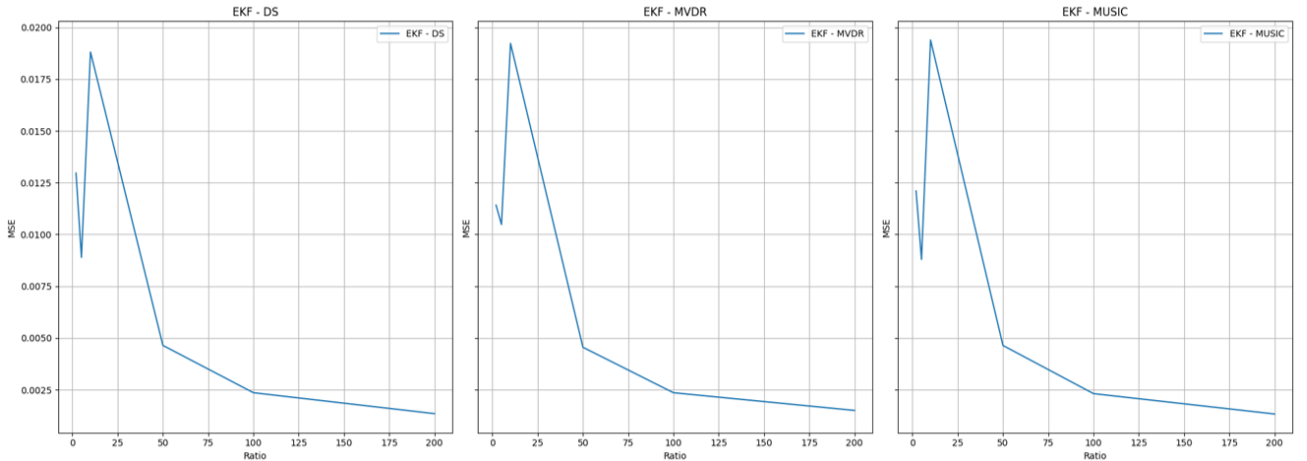


Figure 7.5: Comparison of DOA algorithms

In Figure 7.6, we present the simulation results obtained using the MUSIC algorithm. It can be seen that after a short initial period, the EKF locked onto the pole’s motion characteristics, and from that point onward its estimates became nearly perfect. The estimation ratio used in this simulation was 250.

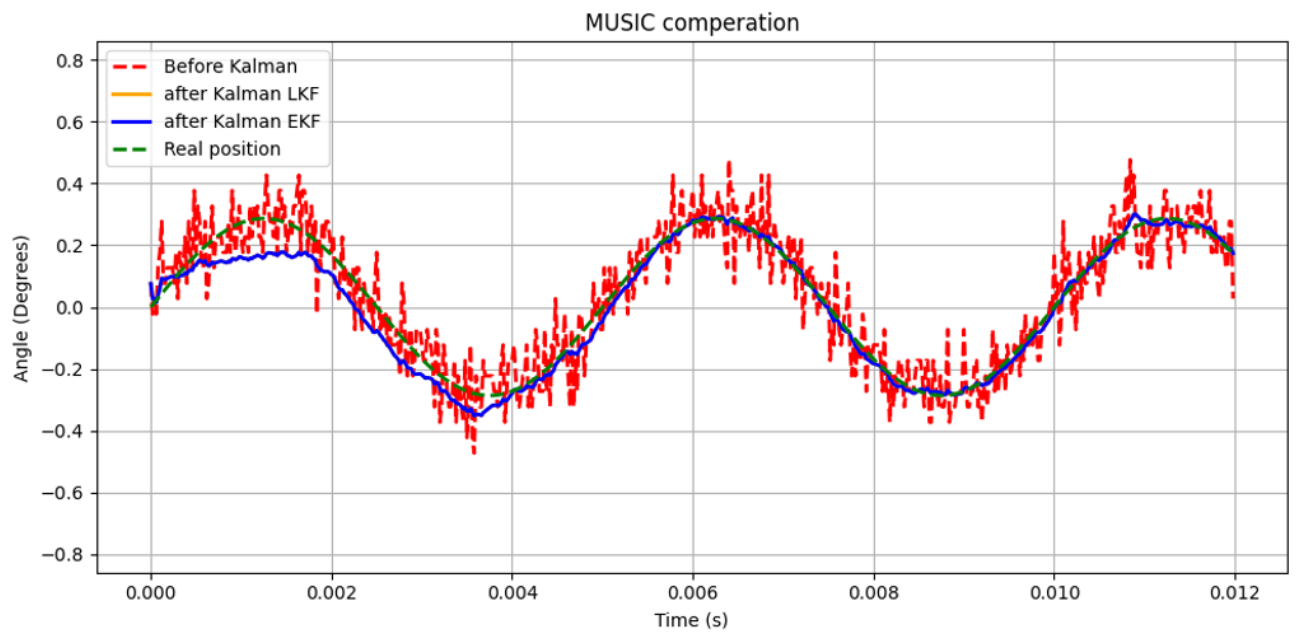


Figure 7.6: MUSIC performance example

Chapter 8

Conclusion

8.1 Our insights

In the previous chapters, we examined the behavior of a DOA system involving estimation using a Kalman filter for a swinging communication pole. From the results we obtained, we can draw the following conclusions:

- **DOA algorithms** — From our simulations, it can be seen that under an SNR of 10, when a single signal is received, there is no noticeable difference in the performance of the tested DOA algorithms. However, we expect that in a real system, under different SNR values and with multiple received signals, differences may become more significant.
- **Minimal sampling ratio** — The ratio between the angle sampling frequency (R matrix generation and DOA activation) and the pole frequency must be sufficiently high to minimize the MSE. That is reasonable due to the fact we use linearization building the EKF formula. We found that a ratio of approximately 50 is adequate for SNR of 10 dB, which corresponds to a sampling frequency of $f_s = 10$ kHz to achieve satisfactory MSE values.
- **Effect of Kalman filtering** — It can be seen that using the EKF formulation improves the MSE of the estimations compared to the true TX position relative to the case without a filter. The LKF we used achieved even better results, but it is limited to a single known frequency, while the EKF estimates the frequency as an internal state. In a real system, where the center oscillation frequency might be unknown, the EKF provides a more practical solution.
- **Calibration** — We demonstrated the importance of calibrating the system to reduce coupling effects.

8.2 For future work

For future work, and in preparation for a real system, we find it important to investigate the following aspects:

- **Handling multiple signals** — Real systems usually consist of multiple propagation paths, IQ imbalance which generates image signals, and may contain several signal sources. To address this, we should examine our system’s response to multiple inputs with different correlation levels. This will allow us to assess potential performance losses in each of the DOA algorithms.
- **Hybrid array adaptation** — In the GHz spectrum, the wavelength is very small, which enables the use of dense antenna arrays. For such arrays, fully digital hardware is impractical. Therefore, our system should be adapted for hybrid arrays, and their performance should be thoroughly investigated.
- **Improved pole model** — As a first approximation, we used a perfect oscillator model. In reality, pole swings are centered around a certain frequency but exhibit more complex behavior. We need to investigate our system — particularly the performance of the LKF and EKF — under more realistic and complex pole swing models.

Bibliography

- [1] Alex Becker. *Kalman filter from the ground up*. Kalmanfilter. net, 2024.
- [2] Peter Delos, Bob Broughton, and Jon Kraft. Phased array antenna patterns-part 1: Linear array beam characteristics and array factor. *Analog Dialogue*, 54(2):1–8, 2020.
- [3] Edno Gentilho Jr, Paulo Rogerio Scalassara, and Taufik Abrão. Direction-of-arrival estimation methods: A performance-complexity tradeoff perspective. *Journal of Signal Processing Systems*, 92(2):239–256, 2020.
- [4] Y. Ginzberg, S. Efimov, and E. Cohen. Mm-wave beamforming through sparse grid gain/phase correction and iq compensation. Department of Electrical and Computer Engineering, Technion, Haifa, Israel, 2025.
- [5] Sooyoung Hur, Taejoon Kim, David J Love, James V Krogmeier, Timothy A Thomas, and Amitava Ghosh. Millimeter wave beamforming for wireless backhaul and access in small cell networks. *IEEE transactions on communications*, 61(10):4391–4403, 2013.
- [6] Marc Lichtman. Beamforming & doa. <https://pysdr.org/content/doa.html>, 2025. Accessed: 2025-08-12.
- [7] Zuo Zhen, Jun Peng Hu, Zhi Ping Huang, and Yi Meng Zhang. Estimation of source number for single-channel received signal. In *MATEC Web of Conferences*, volume 44, page 01064. EDP Sciences, 2016.